**SW Engineering CSC648-848 Fall 2025**

**Project/application title and name:**

**Team Number:** 04

| **Team Members** | **Email** | **Roles** |
|---|---|---|
| Iliana Gallegos | igallegos@sfsu.edu | Team Lead |
| Darien Sngoeun | dsngoeun@sfsu.edu | Back-End Lead |
| Leigh Apotheker | lapotheker@sfsu.edu | Front-End Lead |
| Yuhang Wei | ywei@sfsu.edu | GitHub Maintainer |
| Jonathan Tsang | jtsang1@sfsu.edu | Back-End Dev |
| Roxana Del Toro | rxdt@sfsu.edu | Back-End Dev |
| Megha Rai | mrai@sfsu.edu | Front-End Dev |

**Date**

**Milestone 2 Part I**

**History Table**

| | |
|---|---|
| **Date Submitted For Review** | **11/1/2025** |
| **Date Revised After Feedback** | |

# Table of Contents

# 1. Executive Summary

SFSU Tutor is a tutoring platform for the university. It is designed to connect San Francisco State University students with qualified and verified tutors. The system addresses a key limitation of existing tutoring services, which do not authenticate campus affiliation or align with SFSU's course offerings. Currently, students struggle to find reliable tutors who understand course expectations and tutors face barriers reaching SFSU students. The application provides three main services: course-based search and filtering that allows students to locate tutors by course number, subject, or language preference, tutor profile management where tutors can post availability, hourly rate, and teaching materials subject to administrative approval, and administrative moderation that ensures content and accounts comply with SFSU standards.

All users must register with an @sfsu.edu email address to maintain campus exclusivity. Communication between students and tutors happens through a one-round in-site messaging system to preserve privacy and professionalism. The interface is designed for accessibility and responsiveness across desktop and mobile browsers, and prioritizes  ease of use, data privacy, and trust.

# 2. List of Main Data Items and Entities

Legend

- M/O = Mandatory / Optional
- Enum = controlled list of values

1. **User Account**
    1.1. User ID (M)
    1.2. Full Name (M)
    1.3. SFSU Email (M): must end with @sfsu.edu
    1.4. Role (M, Enum): Student, Tutor, Administrator
    1.5. Account Status (M, Enum): Active, Disabled
    1.6. Created At (M)
    1.7. Last Login At (O)
2. **Tutor Profile**
    2.1. Tutor Profile ID (M)
    2.2. Tutor (User ID) (M): from User Account (Role = Tutor)
    2.3. Display Name (M)
    2.4. Courses Covered (M, List of Course Number)
    2.5. Subject Tags (M, List)
    2.6. Languages Spoken(O, List of Language Name)
    2.7. Hourly Rate (M): display only, there are no in app payments
    2.8. Availability Summary (M)
    2.9. Tutor Profile Photo (O)
    2.10. Sample Material (O)
    2.11. Approval Status (M, Enum): Pending, Approved, Rejected, Removed
    2.12. Visibility (M, Enum): Public, Hidden
    2.13. Created At / Updated At (M)
    2.14. Created By / Updated By (M)
3. **Course Number (SFSU Specific)**
    3.1. Code (M): ex: CSC 648
    3.2. Title (O): ex: Software Engineering
    3.3. Department (O): ex: CSC
    3.4. Notes (O)
4. **Subject Tag**
    4.1. Tag Name (M): ex: Data Structures
    4.2. Description (O)
    4.3. Status (M, Enum): Active, Deprecated
5. **Language**
    5.1. Language Name (M): e.g., English, Japanese, Spanish

5.2. Status (M, Enum): Active, Deprecated

6. **Tutor Profile Photo (Media)**
    6.1. Media ID (M)
    6.2. Owner (Tutor Profile ID) (M)
    6.3. Caption (O)
    6.4. Policy Check (M, Enum): Pending, Approved, Rejected
7. **Sample Material (Media)**
    7.1. Media ID (M)
    7.2. Owner (Tutor Profile ID) (M)
    7.3. Title (O)
    7.4. Type (M, Enum): PDF, Image, Link
    7.5. Policy Check (M, Enum): Pending, Approved, Rejected
8. **In-Site Message**
    8.1. Message ID (M)
    8.2. Sender (User ID) (M): User Account (Role = Student)
    8.3. Recipient (User ID) (M): User Account (Role = Tutor)
    8.4. Message (M)
    8.5. Created At (M)
    8.6. Message Status (M, Enum): Sent, Reported, Removed
    8.7. Linked Report ID (O): Reported Item
9. **Reported Item**
    9.1. Report ID (M)
    9.2. Target Type (M, Enum): Tutor Profile, In-Site Message, User Account
    9.3. Target ID (M)
    9.4. Report Reason (M, Enum): Harassment/Abuse, Inappropriate Content, Spam/Solicitation, Privacy/Safety, Other
    9.5. Details (O)
    9.6. Status (M, Enum): New, Under Review, Resolved, Dismissed
    9.7. Created By (User ID) (M)
    9.8. Created At (M)
    9.9. Resolved By (Admin User ID) (O)
    9.10. Resolved At (O)
    9.11. Resolution Notes (O)
10. **Admin Action**
    10.1. Admin Action ID (M)
    10.2. Action Type (M, Enum): ApproveProfile, RejectProfile, RemoveListing, DisableUser, ReenableUser, RemoveMessage
    10.3. Target Type (M, Enum): Tutor Profile, in-Site Message, User Account
    10.4. Target ID (M)
    10.5. Reason / Notes (O)

10.6. Performed By (Admin User ID) (M)
10.7. Timestamp (M)
10.8. Originating Report ID (O)

**11. Search Panel (Derived View)**

11.1. Course Number (O)
11.2. Subject Tag (O)
11.3. Language Filter (O)

**12. Search Results (Derived View)**

12.1. Tutor Profile ID (M)
12.2. Display Name (M)
12.3. Courses Covered (M)
12.4. Subject Tags (M)
12.5. Languages Spoken (O)
12.6. Hourly Rate (M)
12.7. Approval Status (M): must = Approved

**13. Student Dashboard (Derived View)**

13.1. Message ID (M)
13.2. Recipient Tutor Name (M)
13.3. Recipient Tutor User ID (M)
13.4. Tutor Profile ID (M)
13.5. Created At (M)
13.6. Message Status (M, Enum): Sent, Reported, Removed

**14. Tutor Dashboard (Derived View)**

14.1. Message ID (M)
14.2. Tutor Profile ID (O)
14.3. Sender Student Name (M)
14.4. Sender Student User ID (M)
14.5. Created At (M)
14.6. Message Status (M, Enum): Sent, Reported, Removed
14.7. Profile Status (M): current Approval Status of owned Tutor Profile

**15. Admin Dashboard (Derived View)**

15.1. Moderation Queue (M): list of Pending Tutor Profiles
15.2. Reported Items (M)
15.3. Admin Action Log (M)

# 3. Functional Requirements – Prioritized

Breakdown of the main features: Priority 1 features are the most essential, Priority 2 make the app easier and smoother to use, and Priority 3 features are extra ideas we might add if we have extra time.

**Priority 1 (Must Have)**

**Unregistered Users**

1. Unregistered users shall be able to browse and search tutor profiles without having to log in.

2. Users shall be able to filter search results by course number, subject tag, or language.

3. They shall be able to open and view tutor profiles from the search results.

**Registered Users**

4. Users shall be able to create an account using their @sfsu.edu email address.

5. Users should be able to log in to access features based on their role (student, tutor, or admin).

6. Users should be able to log out to end their session safely.

**Registered Students**

7. Students shall be able to send one in-site message to a tutor through the contact form.

8. Students shall be limited to one message per tutor to keep things simple and avoid spam.

**Registered Tutors**

11. Tutors shall be able to create a profile that includes their courses, subjects, hourly rate, languages spoken, and availability.

12. New or edited tutor profiles shall be marked as “Pending” until an admin reviews them.

13. Tutor profiles should only show up in search results once they’re approved by an admin.

**Administrators**

17. Admins shall be able to review pending tutor profiles and approve or reject them.

18. Admins shall be able to review reported profiles and take action (approve, reject, remove, or disable users).

**System**

23. The system requires users to log in with an @sfsu.edu email before doing anything private.

24. The contact form is only available to verified SFSU users.

**<u>Priority 2 ( preferably desired)</u>**

**Registered Students**

1. Students shall be able to see the messages they’ve sent in their dashboard.

2. Students shall be able to report a tutor profile if something seems wrong or inappropriate.

**Registered Tutors**

3.Tutors shall be able to edit their profiles and resubmit them for review.

4. Tutors shall be able to see messages they’ve received in their dashboard.

5. Tutors shall be able to check the current status of their profile (pending, approved, or rejected).

**Administrators**

6. Admins should be able to remove tutor profiles from public view.

7. Admins shall be able to filter their review lists by profile status (pending, approved, rejected, or removed).

8. Every admin action automatically shall be recorded with a timestamp and details about what was changed.

**System**

9. The system prevents tutors from replying to student messages, keeping the communication one-way for safety and simplicity.(Still thinking if we should keep)

**<u>Priority 3 (Opptional)</u>**

**Administrators**

1. Admins can disable or re-enable a user account if needed.

2. Registered Tutors: Tutors could get simple analytics showing how many students viewed their profile or contacted them (optional future idea).

**Why We Chose These Priorities**

- Priority 1 covers the basic tools the system needs to function for example search, login, messaging, and profile approval.

- Priority 2 includes things that make the app more enjoyable and organized to use like dashboards and filters.

- Priority 3 features are helpful upgrades that aren’t needed for launch but could be added later.

- SFSU-Specific Rule: Requiring an @sfsu.edu email is Priority 1 because it keeps the system limited to SFSU students, tutors, and staff.

# 4. UI Storyboards

**Storyboard 1: Finding and Contacting a Tutor (Student Seeker)**

**Screen 1.1 Home Site**

**SFSU TUTORING PLATFORM**

**Find Your Perfect Tutor**
**Connect with SFSU students and faculty**

**GET STARTED or LOGIN**

**WHY SFSU TUTORING PLATFORM?**
- **Verified SFSU tutors only**
- **Browse by course or subject**
- **Direct messaging system**
- **Flexible scheduling**

**Screen 1.2 Registration Page**

**REGISTER NEW ACCOUNT**

**Email:** [          ]
**(Must use @sfsu.edu email)**

Password:

(Min. 8 characters)

Confirm Password:

First Name:

Last Name:

I am a: ☐ Student seeking tutoring

☐ Tutor offering services

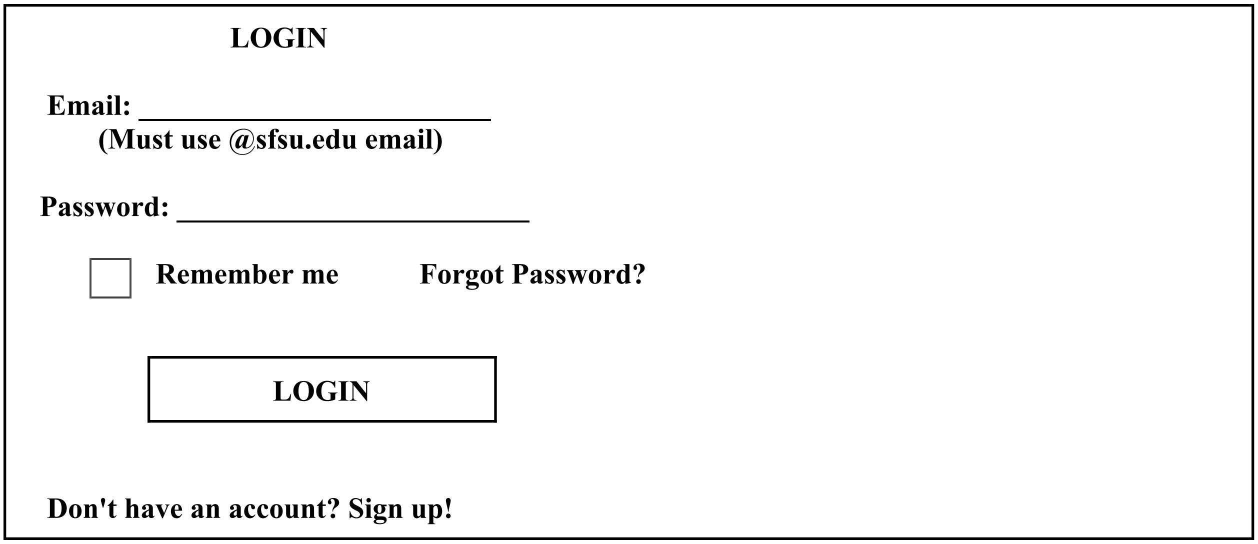
☐ I agree to Terms of Service and Privacy Policy

REGISTER

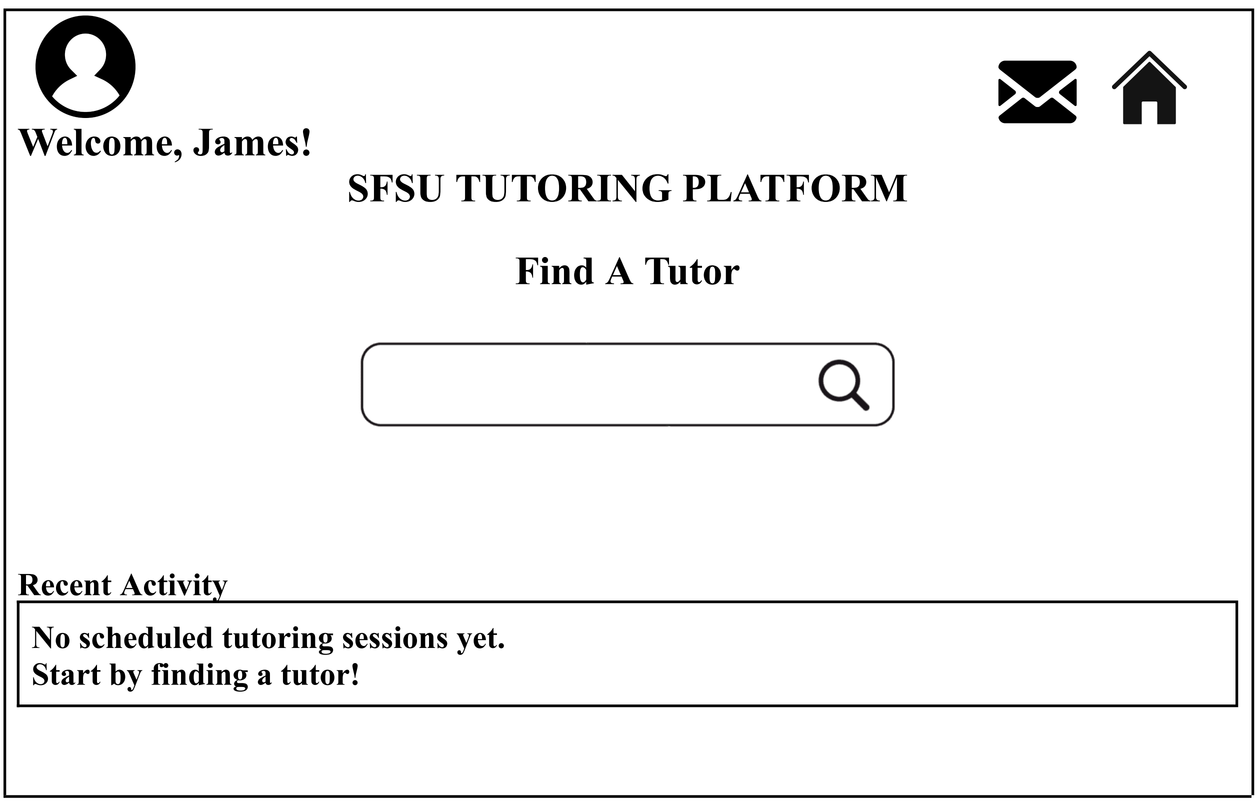
Already have an account? Login

### Screen 1.3 Login Page

SFSU TUTORING PLATFORM




**Screen 1.4 Student Dashboard**




**Screen 1.4 Search Panel with Filters**

## SFSU TUTORING PLATFORM

### Search By

**Subject** [ Computer Science ▼ ]

**Course Number** [ CSC 648 ]

[ SEARCH ] [ CLEAR ]

### Filters

**Hourly Rate: [ ] $0-20 [ ] $21-40 [ ] $41+**

**Availability: [ ] Monday [ ] Tuesday [ ] Mornings [ ] Afternoons**

**[ ] Wednesday [ ] Thursday [ ] Evenings**

**[ ] Friday [ ] Saturday**

**[ ] Sunday**

**Languages: [ ] English [ ] Spanish [ ] Chinese**

[ APPLY FILTERS ]

**Screen 1.5: Search Results**

SFSU TUTORING PLATFORM

**Search Results - "CSC 648"** [Modify Filters]
**Showing 1-2**

**SARAH GARCIA** **$25/hour**
**Graduate Teaching Assistant**
**Courses: CSC 648**
**Languages: English, Spanish**
**Availability Summary: Mon-Fri afternoons, Sat mornings**
VIEW PROFILE

**NAMIKO TURNER** **$30/hour**
**Graduate Teaching Assistant**
**Courses: CSC 648, CSC 317**
**Languages: English**
**Availability Summary: Weekday Evenings**
VIEW PROFILE

**Screen 1.6: Selected Tutor Profile**

< BACK TO RESULTS

**SARAH GARCIA**
**Graduate Teaching Assistant**

**Hourly Rate**
**$25/hour**

**Courses Covered**
**CSC 648 - Software Engineering**

**Languages Spoken**
**English, Spanish**

**Availability Summary**
**Monday-Friday: 2pm-6pm**
**Saturday: 10am-2pm**

**CONTACT**

### Screen 1.7: Contact Form

**FROM:**

**SUBJECT:**

**LANGUAGE REQUEST:**

**MESSAGE:**

**Example: "Hi Sarah, I'm taking CSC 648 this semester and need help with the team project requirements. Are you available for tutoring sessions on Wednesdays?"**

**SEND** **CANCEL**

### Screen 1.8: Return to Dashboard with Confirmed Message

MESSAGE SENT SUCCESSFULLY

✓ Your message has been sent to Sarah Garcia
The tutor will receive your message and can respond through the platform. Check your dashboard for replies.

BACK TO SEARCH | GO TO DASHBOARD

**Storyboard 2: Creating and Managing a Tutor Profile (Tutor)**
**Screen 2.1 Tutor Dashboard**

Welcome, Sarah!

SFSU TUTORING PLATFORM

PROFILE STATUS

YOU DON'T HAVE A TUTOR PROFILE YET
Create your profile to start offering tutoring services.
Your profile will be reviewed by administrators before appearing in search results.

CREATE PROFILE

**Screen 2.2 Tutor Profile Creation/Management Page**

## SFSU TUTORING PLATFORM
### CREATE TUTOR PROFILE

**Display Name:** Sarah Garcia

**Title:** Graduate Teaching Assistant ▼

**Hourly Rate:** $25 ▼ per hour

**Profile Photo:**
[Choose File] No file chosen

**SUBJECTS (Enter topics you can tutor):**
Computer Science ▼ ADD ANOTHER SUBJECT

**COURSES COVERED (Select all that apply):**
Search courses
CSC ▼

CSC 648 - Software Engineering [REMOVE]
CSC 413 - Software Development [ADD]
CSC 317 - Web Development [ADD]

**LANGUAGES SPOKEN:**
[✓] English [✓] Spanish [ ] Chinese [ ] Other

**AVAILABILITY SUMMARY:**
Provide a brief summary of your availability below (days and corresponding times)

Monday - Friday: 2-6 PM
Saturday: 10 AM - 2 PM

BACK SUBMIT FOR REVIEW

**Screen 2.3 Confirmation for Tutor Profile Creation**

BACK TO DASHBOARD

✓ YOUR TUTOR PROFILE HAS BEEN SUBMITTED
Your profile is now under review by our administrators.
This process typically takes 1-2 business days.
You will receive an email notification when your profile is approved or if any changes are needed.
Status: PENDING ADMIN REVIEW

**Screen 2.4 Tutor Dashboard with Profile Approved**

1

**Welcome, Sarah!**

**SFSU TUTORING PLATFORM**

**PROFILE STATUS**

✓ STATUS: ACTIVE
Your profile is now visible in search results!
Approved on: Oct 28, 2025

VIEW PUBLIC PROFILE

**The mockups for admin UI are not included since Workbench will be used.**

# 5. High level Architecture, Database Organization summary only

## Architecture (MVC) Text Summary

- **View:** React pages/components render Search Panel, Search Results, Tutor Profile, Login/ Register, Student Dashboard, Tutor Dashboard. Views only present data
- **Controller:** Express route handlers: Auth, Search, Tutor Profiles, Messaging, Moderation. They validate inputs, enforce roles, and call services

- **Model / Data Access:** Service layer holds business rules (e.g., one-round messaging, visibility/ approval gates). DAO/Repositores do queries and mapping to/from the DB (no business logic)
- **Deployment:** Browser ↔ NGINX(TLS, static) ↔Node/Express ↔ MySQL on AWS EC2. PM2 manages Node process

## Database Organization

Note: Search/Dashboards are derived views (not tables)

- **Core Tables**
    - **user_account:** user_id, full_name, sfsu_email, role, account_status, created_at, last_login_at
    - **tutor_profile:** tutor_profile_id, tutor_user_id, display_name, hourly_rate, availability_summary, approval_status, visibility, created_at, updated_at, created_by, updated_by
    - **course_number:** code, title, department, notes
    - **subject_tag:** tag_name, description, status (if you keep it)
    - **language:** language_name (optionally status if you keep it)
- **Profile lists (link tables)**
    - **tutor_profile_course:** tutor_profile_id, course_code
    - **tutor_profile_tag:** tutor_profile_id, tag_name
    - **tutor_profile_language:** tutor_profile_id, language_name
- **Messaging & Moderation**
    - **in_site_message:** message_id, sender_user_id, recipient_user_id, message, created_at, message_status, linked_report_id
    - **reported_item:** report_id, target_type, target_id, report_reason, details, status, created_by, created_at, resolved_by, resolved_at, resolution_notes
    - **admin_action:** admin_action_id, action_type, target_type, target_id, reason_notes, performed_by, timestamp, originating_report_id
- **Media**
    - **tutor_profile_photo:** media_id, tutor_profile_id, caption, policy_check, file_path, created_at
    - **sample_material:** media_id, tutor_profile_id, title, type, policy_check, file_path_or_url, created_at
- **Constraints**
    - **User_acccount.role** is single-valued: Student, Tutor, Administrator
    - Only **Approved + Public tutor_profile** rows appear in search results
    - One **in_site_message per** (sender_user_id, recipient_user_id) pair (enforced by controller).

## Media Storage

- **Decision:** store images/docs as files on the server, store relative paths in DB.
- **Why:** fast static delivery via NGINX, simpler backups, avoids large DB rows.
- **Policy:** media must have policy_check = Approved before public display. Originals and thumbnails stored in a restricted directory.

## Search/Filter Architecture and Implementation

- **Inputs (from Search Panel):** Either input a Course Number, Subject Tag, or Language Filter. Empty search returns all approved/public profiles
- **Controller/Service Behavior**
    1. Validate filters and normalize whitespace/casing
    2. Compose query with exact filters (IN clauses/joins) and optional text narrowing
    3. Gate by visibility: approval_status='Approved' AND visibility='Public'
    4. Sort by updated_at DESC

# 6. Project Risks and Mitigation

| Risk Type | Description (2–3 lines) | Resolution / Mitigation Plan (2–3 lines) |
|---|---|---|
| **Skills Risk** | Some teammates are still learning advanced React features (hooks, state management) and back-end topics like Express routing and MySQL queries. | Pair senior and junior developers; host short internal workshops and share example code through GitHub Wiki and recorded demos to accelerate learning ramp-up. |
| **Schedule Risk** | M2 deliverables (document + prototype) overlap with other CSC 648 deadlines, leaving limited coordination time. In addition, learning the advanced features may take additional time to get adjusted to. | Use Trello and GitHub Projects to assign clear tasks with deadlines and mid-week check-ins; prioritize MVP features for on-time submission. |
| **Technical Risk** | AWS EC2 deployment and SSL configuration may fail or differ across environments. Database connection strings and firewall rules could cause downtime. | Test deployment early on a staging branch; document setup scripts; assign one back-end engineer (Darien) as DevOps contact for server maintenance. |
| **Teamwork Risk** | Time-zone and schedule differences cause delays in merging and reviewing PRs; merge conflicts happen between front- and back-end branches. | Establish branch-naming conventions (frontend/, backend/, docs/); require pull-request reviews before merging; meet twice weekly via Discord to sync progress. |
| **Content / Legal Risk** | Tutor profiles and sample materials could unintentionally reuse copyrighted or personal data. | Require tutors to agree to a “Content Policy” checkbox; admins will check and regulate uploaded materials for originality; limit visible data to course titles, not personal files. |
| **Integration Risk** | Communication between React front-end and Express API may break due to inconsistent field naming between components and DB tables. | Maintain a shared api-contract.md file describing endpoints and data schema; test integration early in the vertical prototype phase. |

# 7. Project Management

Milestone 2 introduces heavier coordination between design, front-end, and back-end tasks. To manage work efficiently, Team 04 follows an **Agile-style lightweight process**:

- **Task Tracking & Tools**
  The team uses **Github Projects** for visual task boards, for code-level issues.

- **Roles and Responsibilities**

  1. **Team Lead (Iliana Gallegos):** Oversees deadlines, conducts weekly progress checks, ensures document quality.

  2. **Front-End Lead (Leigh Apotheker):** Designs and implements UI storyboards and React components; maintains consistency with Figma mockups.

  3. **Back-End Lead (Darien Sngoeun):** Manages Express routes, MySQL schema, and AWS EC2 deployment; coordinates the vertical prototype.

  4. **GitHub Maintainer (Yuhang Wei):** Handles branching, merging, CI/CD (PM2 deploy + GitHub Actions) and documentation updates.

  5. **Developers (Jonathan, Roxana, Megha):** Implement assigned components, conduct integration testing, and provide peer reviews.

- **Workflow and Schedule**
  Work proceeds in one-week sprints:

  1. **Friday and Monday Meeting:** define weekly goals and assign Trello tasks.

  2. **Mid-Week Checkpoint (Thursday):** brief Discord sync to detect blockers.

  3. **Sunday Wrap-Up:** merge code into the develop branch after peer review.

- **Communication Channels**
  The team mainly uses **Discord** for daily communication and quick questions. All decisions and next-steps are shared on **Discord** as well.

- **Front-End / Back-End Coordination**
  Both sub-teams work semi-independently but align through clearly defined API

contracts. A JSON schema document and Swagger-style endpoint list ensure consistent data exchange.

- **Quality Control & Reviews**
  Every pull request requires at least one review approval. Milestone 2 deliverables are verified by the team lead before submission.

# 8. Use of GenAI Tools for Milestone 2

Tools Used

- **ChatGPT (GPT-5, OpenAI October 2025 version)** – primary tool for brainstorming and document writing
- **GitHub Copilot (Visual Studio Code plugin)** – used for inline code completion and syntax suggestions
- **Figma AI Assistant (beta)** – used for auto-layout and wireframe draft generation

Tasks and Usefulness

| # | Task | GenAI Tool Used | Usefulness | Description / Benefit |
|---|---|---|---|---|
| 1 | Refining Data Dictionary and Entity Definitions | ChatGPT | **High** | Generated clear field descriptions and suggested logical groupings (e.g., linking TutorProfile, Student, and AdminAction entities). This helped ensure consistent naming across the database and API contracts. |
| 2 | Expanding Functional Requirements with Prioritization | ChatGPT | **Medium** | Used ChatGPT to categorize and prioritize requirements by user type and importance. It provided a clean Priority 1–3 table format based on our inputs from M1. |
| 3 | Drafting UI Storyboards and Wireframe Captions | Figma AI Assistant + ChatGPT | **High** | Figma AI helped auto-layout the low-fidelity wireframes for Search, Tutor Profile, and Dashboard pages. ChatGPT assisted in writing short, user-oriented annotations for each mockup screen. |
| 4 | Writing Section 6 (Project Risks) and 7 (Project Management) | ChatGPT | **High** | Used to structure the risk table and project workflow summary in professional report language. Final text was edited and validated by the team lead (Iliana Gallegos). |

| | | | | |
|---|---|---|---|---|
| 5 | Backend Prototype Coding Support (Express + MySQL) | GitHub Copilot | **Medium** | Helped autofill boilerplate code for Express routes, SQL connection setup, and error handlers. Team members reviewed and manually tested all AI-generated snippets. |
| 6 | Documentation Consistency and Proofreading | ChatGPT | **High** | Provided grammar and structure checks for the entire Milestone 2 document to ensure clarity and consistency with the Milestone 1 tone. |

Examples of Prompts Used

**Prompt Example 1:**

> "Expand the Tutor entity from Milestone 1 into a detailed data dictionary table with field names, descriptions, and mandatory/optional tags for Milestone 2 document."

**Prompt Example 2:**

> "Write a professional risk analysis section for a university tutoring web application built with Node.js, React, and MySQL, categorizing risks as skills, schedule, technical, and teamwork."

**Prompt Example 3:**

> "Generate short black-and-white UI wireframe captions for 'Search Tutor' and 'Student Dashboard' pages focusing on clarity and navigation flow."

**Prompt Example 4:**

> "Suggest a clean Priority 1–3 layout for functional requirements grouped by user type, matching CSC 648 Milestone 2 guidelines."

Overall Reflection

Using GenAI tools greatly improved our workflow efficiency and clarity of deliverables:

- **ChatGPT** was most helpful for writing, editing, and expanding non-technical sections such as requirements and risk analysis.

- **Copilot** sped up prototype coding and helped newer team members learn Node.js syntax faster.

- **Figma AI** made UI wireframe creation smoother by automatically balancing layout and text alignment.

The team understood that GenAI is a collaborative assistant — not a replacement for original work. All outputs were checked, tested, and edited by team members for accuracy and authenticity. This practice helped maintain academic integrity and produced a professional document aligned with CSC 648 expectations.

# 9. Team Lead Checklist

For each item below team lead must answer with only one of the following: DONE/OK; or ON TRACK (meaning it will be done on time, and no issues perceived); or ISSUE (you have some problems, and then define what is the problem with 1-3 lines)

• So far all team members are fully engaged and attending team sessions when required

- On Track

• Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing

- Done

• Team reviewed suggested resources before drafting Milestone 2

- Done

• Team lead checked Milestone 2 document for quality, completeness,formatting and compliance with instructions before the submission

- Done

• Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission

- Done

• Team shared and discussed experience with GenAI tools among themselves

- Done